Technical details

This document describes the function that returns the earliest longest strictly increasing run, the single left-to-right scan that finds it, and the console program that calls it.

Problem

`Find` takes one string of integers separated by single spaces and returns one string. The result is the contiguous stretch that is strictly increasing, is at least as long as every other such stretch, and starts earliest when lengths are equal. A single value is a run of length one. An empty string returns an empty string.

Comparison is numeric and culture-invariant. `-5 -1 0 4` increases. `9` is not greater than `10`. An equal neighbour ends the run. The step may be any positive amount, so `1 100 101` is one run.

A run is not a subsequence that skips values

The classic longest increasing subsequence may skip values and is a different problem, usually solved in `O(n log n)` time. That answer is rejected here. For `6 2 4 6 1 5 9 2`, both `2 4 5 9` and `2 4 6 9` are longer than `2 4 6`, and both skip a value that sits between the chosen numbers. The function returns `2 4 6`.

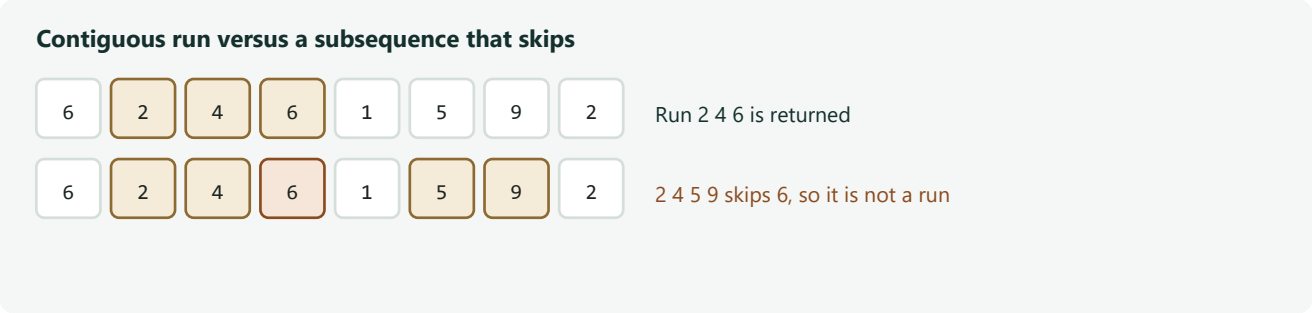


Figure 1. Gold cells are kept. Clay cells are the values a longer subsequence would have to skip.

Components

The library method is `LongestIncreasingRunFinder.Find`. It rejects null with `ArgumentNullException`, then runs three private steps. Parsing and formatting stay private. Callers pass one string and receive one string.

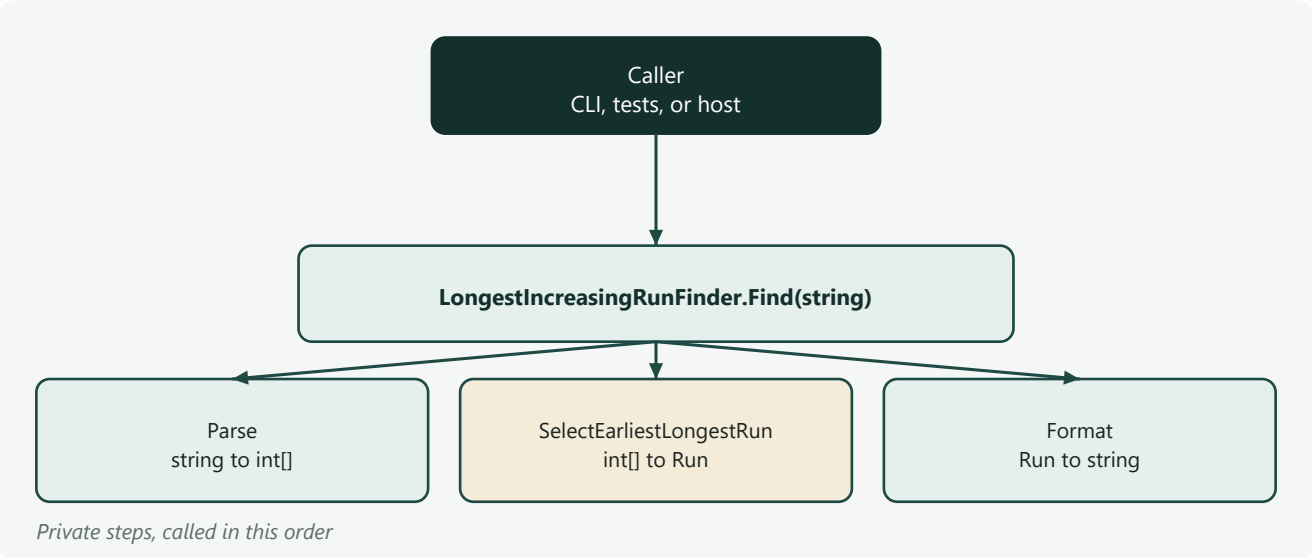


Figure 2. Call path from a host into the three private steps.

Step	Responsibility
`Parse`	Split on a single space and parse each token with CultureInfo.InvariantCulture. An empty string becomes an empty array.
`SelectEarliestLongestRun`	One forward pass. Record the start index and length of the earliest longest maximal run.

Step	Responsibility
`Format`	Write values[start .. start+length) separated by one space. Length zero returns an empty string. No leading or trailing space.

Selection algorithm

A maximal run is already ended by the first value that is not strictly greater than its predecessor. The scan therefore does not search every pair of endpoints. It walks the array once, closes a run when the increase stops, and replaces the recorded answer only when the closed run is strictly longer. Equal length leaves the earlier start in place.

- An empty array returns `Run(0, 0)`.
- Otherwise the first value is a run of length 1, so `bestLength` starts at 1.
- Inside the loop, `index` is the first value that does not extend the current run. Its length is `index - runStart`.
- The final run has no following value, so it is closed once after the loop with the same length test.
- The search performs `n - 1` comparisons. Every value is read, which matches the lower bound: an unread value could extend a run or start a longer one.

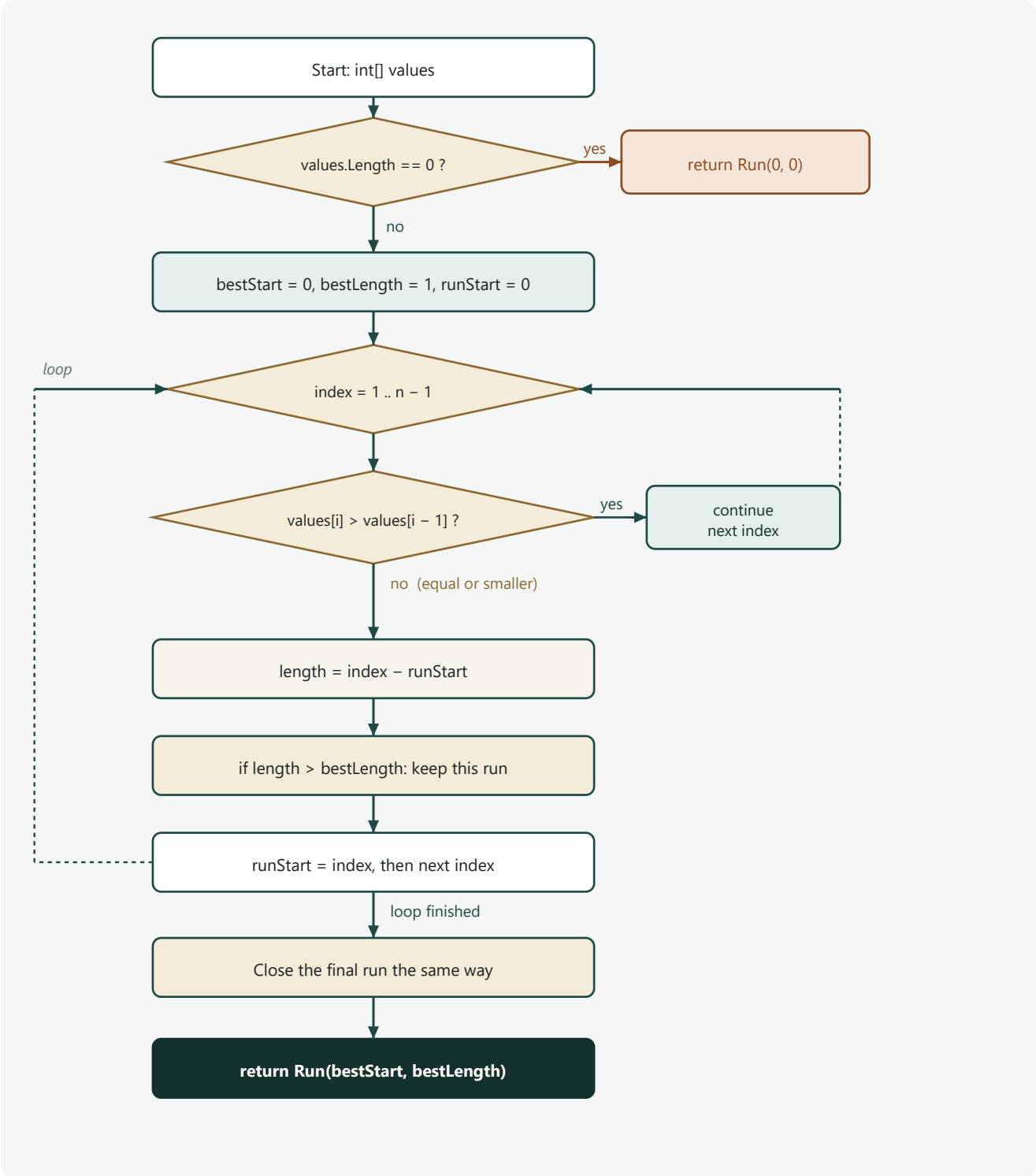


Figure 3. Control flow of SelectEarliestLongestRun. The dashed line returns to the next index.

State while scanning the tie

Input 6 2 4 6 1 5 9 2. Indexes are zero-based. A new run replaces the best run only when its length is greater. The second length-3 run does not replace the first.

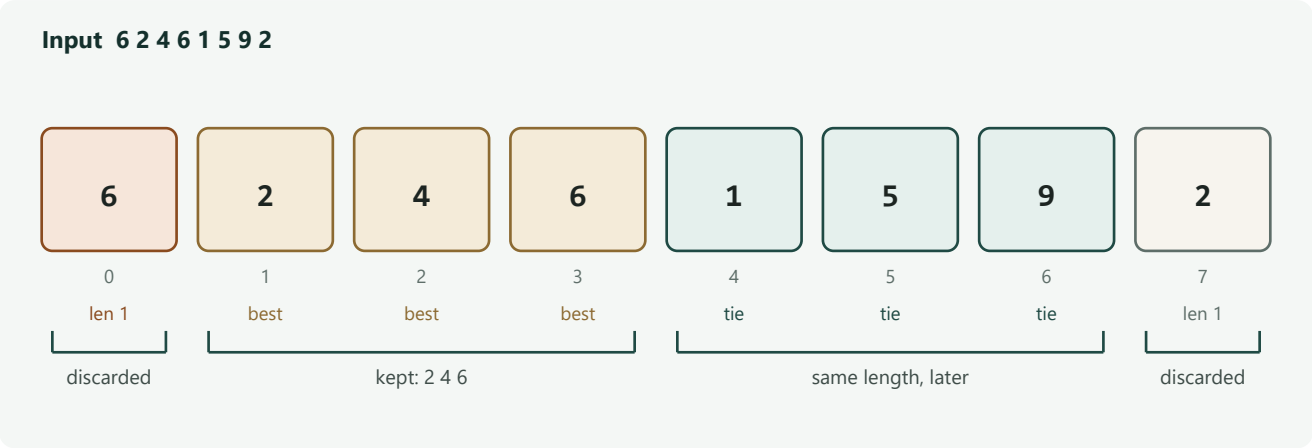


Figure 4. The four maximal runs. The gold run is the one returned.

Event	Closed run	Length	bestStart	bestLength	Recorded answer
`Start at 6`	—	—	0	1	6
`2 does not increase`	6	1	0	1	6 (equal length, keep)
`1 does not increase`	2 4 6	3	1	3	2 4 6
`2 does not increase`	1 5 9	3	1	3	2 4 6 (tie, keep earlier)
`Close the final run`	2	1	1	3	2 4 6

Table 1. Recorded answer after each time a run is closed.

The shorter published example

`6 1 5 9 2` has maximal runs 6, `1 5 9`, and 2. Length 3 replaces length 1, and the final length 1 does not replace it. The result is `1 5 9`.

Cost

Resource	Bound	Where it is spent
`Time`	O(n)	n − 1 numeric comparisons, plus a linear parse and a linear format of the chosen run.
`Extra memory`	O(n)	The integer array. The search itself stores a few indexes: bestStart, bestLength, runStart, and the loop index.
`Output`	O(k)	k is the length of the chosen run. Formatting writes only that span.

A variant can avoid the integer array by remembering the character span of the best run and returning that slice of the original string. The indexes stay a handful of integers. `Find` uses the array form so the tie is visible as value indexes rather than character offsets.

Console program

`LongestIncreasingRun.Cli` is a thin host. One command-line argument is the whole sequence, including its spaces, and the process prints the raw run and exits. With no argument it prompts on standard error, reads one line, and repeats. An empty line, or the end of input, exits with code 0. More than one argument prints usage and exits with code 1.

Interactive output is prefixed. The printed line is `Longest increasing subsequence is:` followed by the run. The one-argument path does not add that prefix. The library method itself returns only the run.

The solution targets `net9.0`. `global.json` pins SDK 9.0.319 and allows a newer 9.x SDK. The container build uses `mcr.microsoft.com/dotnet/sdk:9.0` and `mcr.microsoft.com/dotnet/runtime:9.0`.

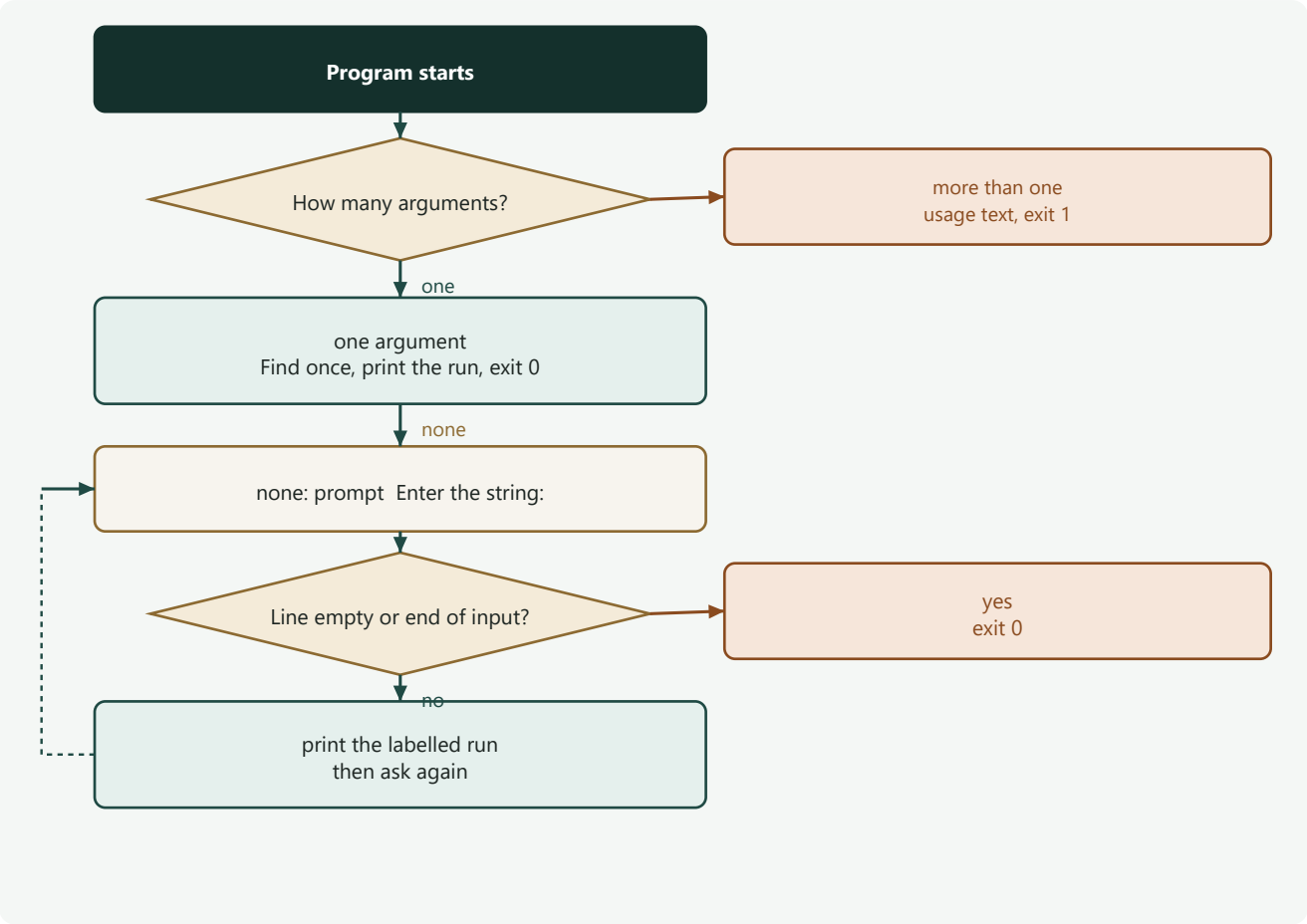


Figure 5. Console control flow. The dashed arrow returns to the prompt.

Project map

Path	Role
`src/LongestIncreasingRun/LongestIncreasingRunFinder.cs`	Find, Parse, SelectEarliestLongestRun, and Format.
`src/LongestIncreasingRun.Cli/Program.cs`	Argument handling, the prompt loop, and printing.
`tests/LongestIncreasingRun.Tests/RunRulesTests.cs`	One case per rule, plus a brute-force oracle over random sequences.
`tests/LongestIncreasingRun.Tests/AcceptanceFixtureTests.cs`	Acceptance cases AT-01 through AT-11 and RC-01 through RC-08.
`Downloads/acceptance-fixtures.json`	Exact inputs, expected strings, and SHA-256 digests loaded by the acceptance tests.

What the tests lock down

- A single value is returned unchanged. A strictly increasing input is returned unchanged.
- A strictly decreasing input returns the first value, because every later run has length 1.
- Equal neighbours end the run: `1 2 2 3` returns `1 2`.
- Equal lengths keep the earlier start: `1 3 2 4` returns `1 3`.
- Negative numbers and zero compare numerically.
- Null throws `ArgumentNullException`. An empty string returns an empty string.
- Two hundred random sequences of length 1 to 31 must match a brute-force scan of every maximal run.